From Euclidean to Minkowski: How Distance Shapes Machine Learning

“What makes distance metrics so important in machine learning and data analysis? How do Euclidean, Manhattan, and Minkowski distances differ visually and mathematically? And more importantly, which one should you choose for your task?”

In this article, we’ll dive deep into these distance metrics, starting with their geometric interpretations. The contour plots above (Fig 1) provide an intuitive understanding of how each metric measures distance from a reference point. By the end, you’ll have a clear understanding of when and why to use each of them.

Euclidean Distance, *the straight-line measure much applied in clustering and k-NN algorithms.*

Manhattan Distance *sums absolute differences, thus, are best for grid-based and high-dimensional spaces.*

Minkowski Distance*, a generalized form that combines both Euclidean and Manhattan distances.*

With an understanding of these metrics, we will get a glimpse of how machines see similarity and take data-driven decisions. Let’s roll!

1.Euclidean Distance

It is the most commonly used distance metric in machine learning. It measures the straight-line (or “as-the-crow-flies”) distance between two points in a space. It’s easy to understand because it follows the same concept as measuring the distance between two points on a flat plane.

Mathematical Formula

For two points, $P_1(x_1, y_1)$ and $P_2(x_2, y_2)$ in a 2-dimensional space, the Euclidean distance between them is given by the formula:

$$d(P_1, P_2) = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

In higher dimensions the formula generalizes as,

$$d(P_1, P_2) = \sqrt{\sum_{i=1}^{n} (x_i - y_i)^2}$$

Where:

- $P_1(x_1, x_2,\ldots,x_n)$ and $P_2(y_1, y_2,\ldots,y_n)$ the two points in an n-dimensional space.
- The distance is calculated by squaring the differences in each dimension, summing them up, and taking the square root of that sum.

Example Calculation

For two points *P1(3,4)* and *P2(7,1)* the calculation for euclidean distance is,

$$d(P_1, P_2) = \sqrt{(7-3)^2 + (1-4)^2} = \sqrt{16+9} = \sqrt{25} = 5$$

Therefore, the Euclidean distance between these two points is 5.

1.a A machine learning use case example: k-Nearest Neighbors (k-NN)

The k-NN algorithm classifies a new data point based on the class of its closest neighbors.

Example: Classifying an Image Using k-NN

Suppose we have three images represented by feature vectors in a **2D space**:

Image	Feature 1 (X)	Feature 2 (Y)	Class
A	1	2	Cat
B	4	5	Dog
C	7	8	Dog
New	3	3	?

We want to classify the **New Image (3,3)** using k-NN with k=2, meaning we find the two nearest points.

1st : Compute Euclidean Distances

Using the formula:

- Distance to **A (1,2)**:

$$d = \sqrt{(3-1)^2 + (3-2)^2} = \sqrt{4+1} = \sqrt{5} \approx 2.24$$

- Distance to **B (4,5)**:

$$d = \sqrt{(3-4)^2 + (3-5)^2} = \sqrt{1+4} = \sqrt{5} \approx 2.24$$

- Distance to **C (7,8)**:

$$d = \sqrt{(3-7)^2 + (3-8)^2} = \sqrt{16+25} = \sqrt{41} \approx 6.4$$

2nd : Find the Nearest Neighbors

The **two nearest** points are:

A (Cat) → Distance 2.24

B (Dog) → Distance 2.24

3rd : Assign the Class

If we use **k = 3**, the new image is classified as **Dog**. (the nearest three points are **A (Cat), B (Dog), and C (Dog)**. Since **Dog appears twice**, the new image would be classified as **Dog**.)
If we stick to **k = 2**, a tiebreaker rule must be applied! So, depending on tie-breaking rules, **the new image could be either Cat or Dog**.

Handling Ties in k-NN Classification

Ties can occur in k-NN when there's an equal number of nearest neighbors from different classes. Here are some common ways to handle ties:

Increase k *→ A larger k often reduces ties since more neighbors are considered. Choosing an odd value for k (e.g., k = 3, 5) helps prevent equal splits.*

Weighted Voting *→ Assign higher weights to closer neighbors. For example, use* ***1/distance*** *as a weight so that closer points contribute more to the decision.*

Prior Class Probability *→ If the dataset is imbalanced, prefer the class that appears more frequently in the training set.*

Random Tie-Breaking *→ If all else fails, assign the class randomly when a tie occurs. This is less ideal but sometimes necessary.*

Applying Weighted Voting to the Example:

Instead of a simple count, we assign weights based on **1/distance**:

- A (Cat) → **Weight = 1 / 2.24 ≈ 0.446**
- B (Dog) → **Weight = 1 / 2.24 ≈ 0.446**
- C (Dog) → **Weight = 1 / 6.4 ≈ 0.156**

Summing up the weights:

- **Cat: 0.446**
- **Dog: 0.446 + 0.156 = 0.602**

Since Dog has a higher weighted score, the image is classified as **Dog**!

1.b When to Apply Euclidean Distance

Euclidean distance is a great choice when:

- **Data is continuous**: It works well when the features of your data can take any value along a continuous scale (e.g., height, weight, age).
- **You want to measure straight-line similarity**: If the relationship between data points can be captured by direct distances without considering more complex interactions between features.
- **Data is relatively small-dimensional**: In low-dimensional spaces, Euclidean distance works effectively to measure proximity.

1.c Limitations of Euclidean Distance:

- **Sensitive to Scale:** If features have different ranges (e.g., height vs. age), one can dominate the distance calculation. **Solution:** Use feature scaling (normalization or standardization).

- **High-Dimensional Issues:** In many dimensions ("curse of dimensionality"), distances become less meaningful, making clustering harder.
- **Assumes Linearity:** It assumes a straight-line relationship, which may not hold for non-linear data.

2. Manhattan Distance

Manhattan Distance, also called **Taxicab Distance** or **L1 Distance**, measures the distance between two points by **only moving horizontally and vertically** — like how a taxi navigates city blocks.

Formula for Manhattan Distance

For two points **P1(x1, y1)** and **P2(x2, y2)** in a 2D space:

$$d(P_1, P_2) = |x_2 - x_1| + |y_2 - y_1|$$

For **n-dimensional** space:

$$d(P_1, P_2) = \sum_{i=1}^{n} |x_i - y_i|$$

Example calculation,

Let's calculate the Manhattan distance between points **A(2, 3)** and **B(5, 7)**:

$$d(A, B) = |5 - 2| + |7 - 3|$$

$$= |3| + |4| = 3 + 4 = 7$$

The Manhattan distance between A and B is 7.

2.a Manhattan Distance for Word Frequency in NLP

In text analysis, words are counted and represented as frequency vectors. Manhattan distance measures how different two texts are by summing the absolute differences in word counts.

For example, if two sentences have word frequencies:

- Text 1: {“apple”: 3, “banana”: 1}
- Text 2: {“apple”: 1, “banana”: 2}

The Manhattan distance is: |3−1| + |1−2| = 2+1 =3

A smaller distance means similar word usage, while a larger distance means more differences. This is useful for comparing documents in NLP.

Use Case in NLP

Document Similarity *:Helps compare text documents based on word usage.*
Spam Detection *: Identifies emails with similar word patterns.*

Plagiarism Detection *: Measures text similarity without needing deep understanding.*

2.b Limitations of Manhattan Distance (L1 Norm)

· No Diagonal Movement — Only considers horizontal and vertical paths, making it inefficient for real-world distance measurements.

· Sensitive to Axis Alignment — Works best in grid-like spaces but fails when features are correlated.

· Not Ideal for High Dimensions — Becomes less meaningful as dimensions increase due to the curse of dimensionality.

· Feature Scaling Impact — Requires normalized data; otherwise, features with larger scales dominate.

· Less Robust to Outliers — Absolute differences can still be affected by extreme values.

3. Minkowski Distance

The **Minkowski Distance** is a generalized distance metric that includes both Euclidean and Manhattan distances as special cases. It's defined as:

Press enter or click to view image in full size

$$d(P_1, P_2) = \left(\sum_{i=1}^{n} |x_i - y_i|^p \right)^{1/p}$$

- $P_1(x_1, x_2,\ldots,x_n)$ and $P_2(y_1, y_2,\ldots,y_n)$ are two points in n-dimensional space.

- p is a parameter that determines the "type" of distance:
- When $p = 1$, this is Manhattan distance.
- When $p = 2$, this is Euclidean distance.
- As $p \to \infty$, it becomes the Chebyshev distance.

3.a Minkowski Distance for p>2:

For p>2, the Minkowski distance becomes more sensitive to large differences between coordinates.

Effect of increasing *p*: As p increases, the largest differences dominate the distance calculation. This leads to a "stretching" effect where points that differ in one or more coordinates by a large amount are considered farther apart.

Example:

point_1 = [1,2,3]

point_2 = [1,3,10]

- The first two coordinates are **similar**.
- The **third coordinate has a large difference**: $|3-10|=7$

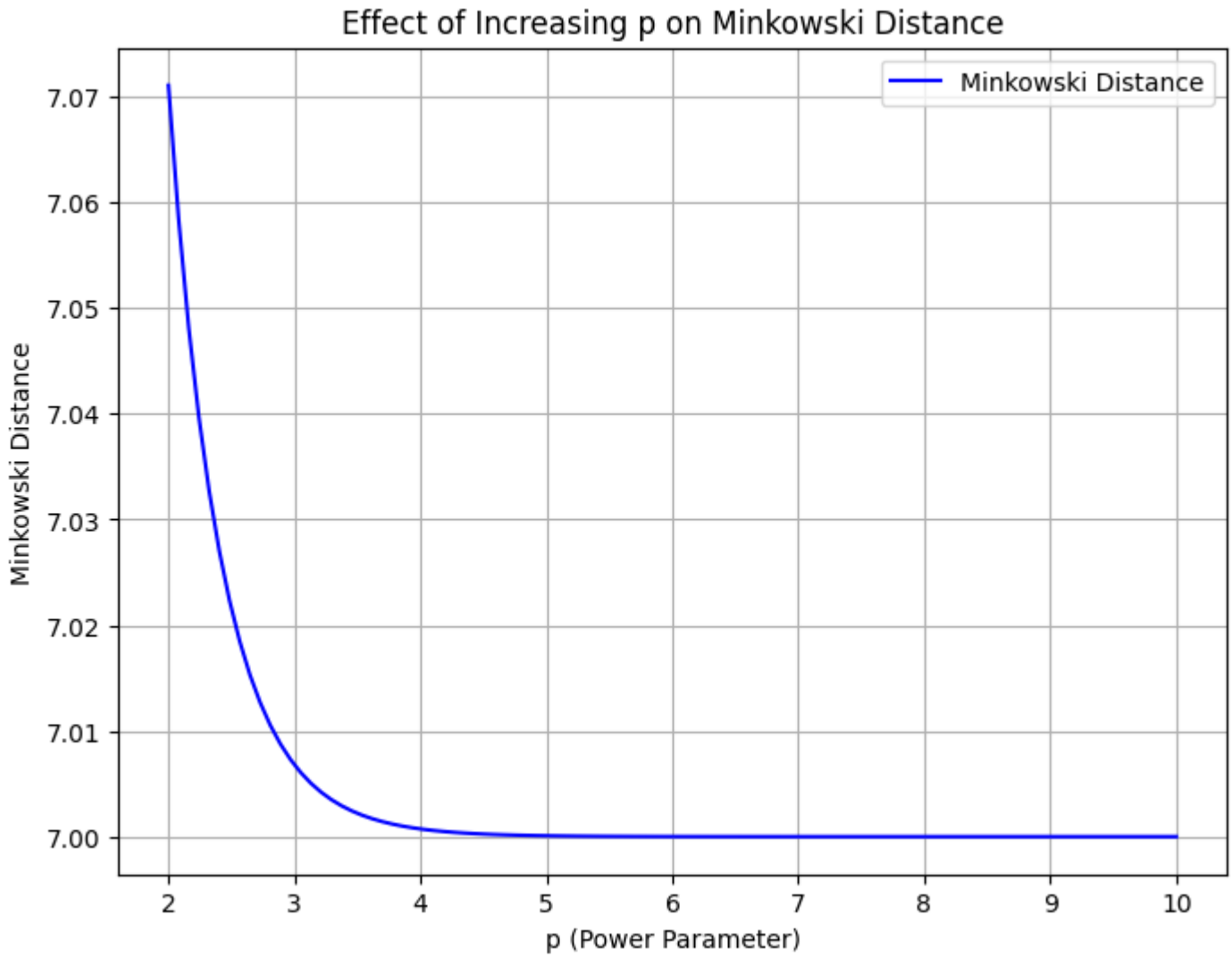

Fig 3: Effect of Power Parameter on Minkowski Distance".

- **When p=2:** The metric is **fair to all dimensions** (Euclidean distance).
- **When p>2: The largest coordinate differences dominate**.
- **When p→∞:** Minkowski distance approaches *Chebyshev distance*, where only the **largest difference matters**.

Having in mind that,

- p=1 (Manhattan Distance): Best for grid-like or axis-aligned data where each feature is independent.

- p=2(Euclidean Distance): Commonly used for continuous data where relationships between features are important.
- p>2: Useful for tasks where large deviations in one feature dominate the distance metric.
- p→∞: Focuses on the largest deviations, ideal for anomaly detection or tasks emphasizing extreme cases.

here is a tabulated form of ML applications

ML Application	Value of p	Description	Example
k-Nearest Neighbors (k-NN)	p=1(Manhattan Distance)	Best for grid-like structures (e.g., images, geographical data with axis-aligned movement). Useful when features contribute independently.	**Example:** Image or geographical data.
	p=2(Euclidean Distance)	Most common for general problems with continuous data (e.g., image classification, disease prediction).	**Example:** Predicting whether a person will buy a product based on age and income.
	p>2p	Penalizes outliers more heavily, highlighting extreme cases (e.g., anomaly or fraud detection).	**Example:** Fraud detection, emphasizing large feature differences.
Clustering Algorithms	p=1(Manhattan Distance)	Helps form diamond-shaped clusters, suitable for grid-like or axis-aligned data.	**Example:** Data clustering based on age and income.
	p=2(Euclidean Distance)	Common in k-Means, forms spherical clusters, used for continuous data with "straight-line" distance.	**Example:** Clustering customers based on characteristics like income and age.
	p>2	Large p values make the clustering sensitive to deviations in one feature, useful when features dominate.	**Example:** Identifying key characteristics of specific customer segments.
Anomaly Detection	p=1(Manhattan Distance)	Measures total deviation per feature, especially useful for sparse or categorical features (e.g., credit card transactions).	**Example:** Fraud detection, where deviations in transaction amount or location are important.
	p=2(Euclidean Distance)	Sensitive measure of overall deviation, suitable for continuous data.	**Example:** Detecting outliers in a continuous dataset like transaction amounts.
	p→∞	Focuses on the largest difference between any two features, highlighting extreme cases in one dimension.	**Example:** Detecting unusual fraud where one feature (e.g., transaction amount) deviates drastically.
Natural Language Processing (Word Frequency)	p=1(Manhattan Distance)	Captures absolute differences in word occurrences, useful in low-dimensional vector spaces.	**Example:** Identifying document similarity based on word frequency in a short document or small dataset.
	p=2(Euclidean Distance)	Provides a smooth, continuous measure of similarity based on all word occurrences.	**Example:** Document classification based on word frequencies in a larger set of documents.
	p>2	Focuses more on large differences between features (words), emphasizing outliers.	**Example:** Detecting documents with drastically different topics by focusing on key word differences.

Table 1: Overview of Machine Learning Applications with Different Minkowski Distance Metrics (p) and Their Practical Use Cases

3.b Limitations of Minkowski distance:

- **Sensitive to Large Differences** — Higher *p* values exaggerate large coordinate differences, distorting distance measurements.

- **Computationally Expensive** — Raising values to the power of p and taking roots is slow for high-dimensional data.
- **Requires Normalization** — Unequal feature scales can dominate the distance calculation, leading to misleading results.

From Euclidean to Manhattan to Minkowski, we've uncovered how each distance metric serves a unique purpose in machine learning. These metrics aren't just abstract mathematical tools — they define how algorithms perceive similarity and make decisions.

Ultimately, the choice of distance metric depends on your dataset's structure and the problem at hand. As we saw in Fig 1 where,

Euclidean Distance *forms circular contours, representing straight-line distance in continuous space*

Manhattan Distance *creates diamond-shaped contours, as movement is restricted to horizontal and vertical directions*

Minkowski Distance *acts as a bridge between Euclidean and Manhattan distances, producing rounded contours that emphasize coordinate differences.,*

these metrics offer different perspectives on proximity, shaping outcomes in machine learning models. By understanding these geometric interpretations, we gain insight into how machines "see" data, helping us make more informed decisions and build smarter models. So, whether you're navigating straight-line spaces or taking grid-like steps, remember — distance truly matters in machine learning.

Refrences

Distance Metrics: Euclidean, Manhattan, Minkowski, Oh My! - KDnuggets

Distance Metrics | Understand Distance Metric in Machine Learning

From Euclidean to Minkowski: How Distance Shapes Machine Learning | by Kezya Steffyn | Medium

Distance Measure Types in Machine Learning - Dot Net Tutorials

https://www.researchgate.net/publication/243573998_Introductory_functzonal_analysis_with_applicatzons?_tp=eyJjb250ZXh0Ijp7ImZpcnN0UGFnZSI6InB1YmxpY2F0aW9uIiwicGFnZSI6InB1YmxpY2F0aW9uIn19

https://www.researchgate.net/publication/376257841_Distance_Metrics_for_Machine_Learning_and_it's_Relation_with_Other_Distances